

Modeling Collisions in Python: From Elastic to Inelastic to Plain



[Rhett Allain](#)

Follow

8 min read

.

1 day ago

Listen

Share

More

Press enter or click to view image in full size

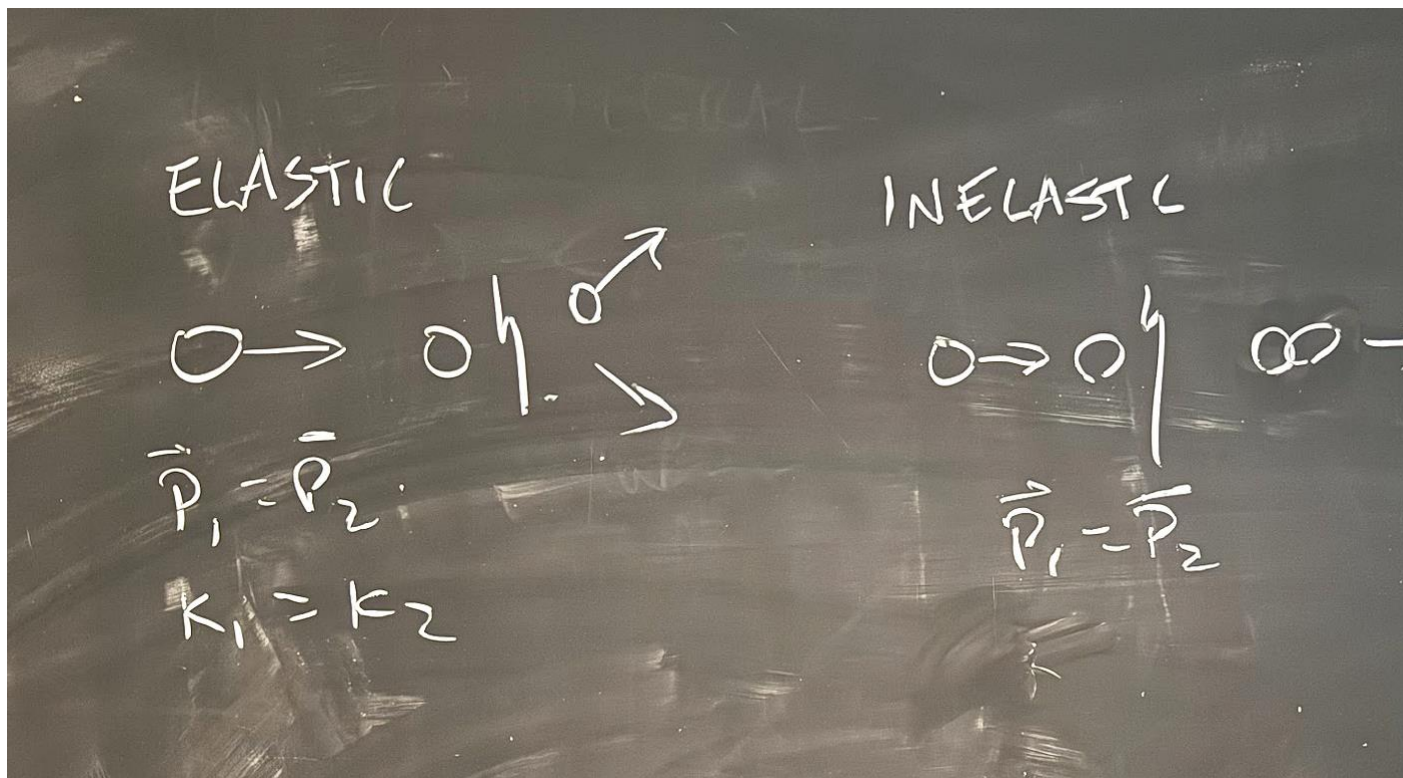


Photo: Rhett Allain. Three types of collisions

In your introductory physics course, there are three types of collisions. The simplest (from a physics perspective) is the inelastic collision in which two objects impact and then stick together. The next kind is the elastic collision. This is a “bouncy” collision. Finally, there is a collision that is in between the elastic and inelastic. I’m just going to call these “plain” collisions.

How do you model these three types of collisions in python? It’s not too difficult, but there are some tricks that I need to show you to get it working.

The Physics of Collisions

Before getting to numerical models, let’s just go over the very basic physics stuff for collisions. Imagine I have two balls (I will call them A and B) that collide. During this collision, there is a force that ball A exerts on ball B. However, Newton’s 3rd law says that forces come in pairs. So, if A pushes on B then B pushes back on A with the same magnitude (but in the opposite direction).

$$\vec{F}_{B-A} = - \vec{F}_{A-B}$$

If that’s the only force on each of the balls, then the momentum principle says the following:

Press enter or click to view image in full size

$$\vec{F}_{B-A} = \frac{\Delta \vec{p}_A}{\Delta t} \qquad \vec{F}_{A-B} = \frac{\Delta \vec{p}_B}{\Delta t} = - \vec{F}_{B-A}$$

Where we define the momentum (p) as the product of mass (m) and velocity (v). Since the time interval for the collision is the same for both balls — the change in momentum for A must be equal and opposite the change in momentum for B. Or, you could say the total momentum before the collision is equal to the total momentum after the collision. We call this conservation of momentum.

Just a quick reflection here. Momentum is conserved only because the forces on the balls are equal and opposite. If there is some extra force on one of the balls, this is no longer true. However as long as the forces are equal and opposite — we are good.

This means that it doesn't matter what kind of collision we have — momentum will be conserved (yes, with no external forces).

$$\vec{p}_{A1} + \vec{p}_{B1} = \vec{p}_{A2} + \vec{p}_{B2}$$

In the special case of a perfectly elastic collision, kinetic energy is also conserved.

Press enter or click to view image in full size

$$K_{A1} + K_{B1} = K_{A2} + K_{B2}$$

$$\frac{1}{2}m_A v_{A1}^2 + \frac{1}{2}m_B v_{B1}^2 = \frac{1}{2}m_A v_{A2}^2 + \frac{1}{2}m_B v_{B2}^2$$

That's the physics. Now for some python.

Elastic Collisions

Let's start with the easiest collision — the elastic collision. Wait. Did I say inelastic was the simplest? Yes, that's the easiest if you are going to use physics to find the final velocities (since they stick together). However, it's more complicated to model that with python.

What is a numerical model? Instead of finding final velocities of objects, we are going to model the motion in short time intervals. If you know the force on an object, we can use that force to change that object's momentum. Once you know the momentum, you can then use that to change the position. Then, we can just repeat this process as many times as we like to get the motion of an object.

This is what those calculations look like.

$$\vec{p}_2 = \vec{p}_1 + \vec{F} \Delta t$$

$$\vec{r}_2 = \vec{r}_1 + \frac{\vec{p}}{m} \Delta t$$

Here p_2 is the momentum at the end of the short time interval and p_1 is at the beginning. Likewise, r_2 is the final position and r_1 is the initial position. Notice that this is only approximately true — but with a small time interval, it works fairly well.

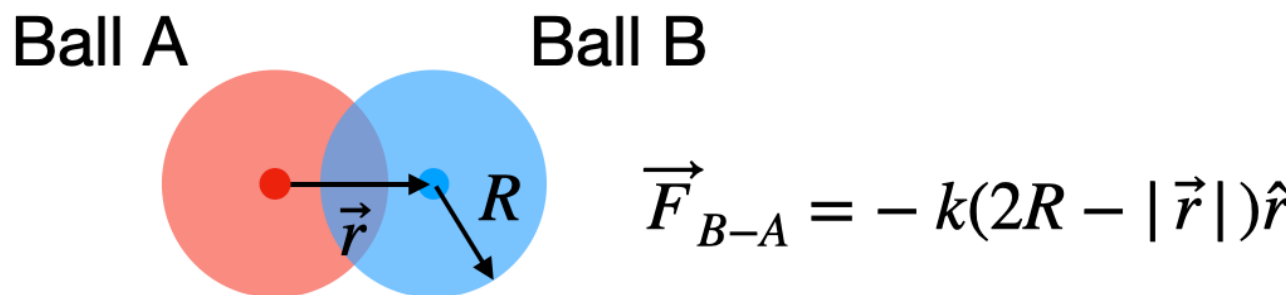
If you have a time interval of 0.01 seconds, you would have to do these calculations 100 times just to get 1 second of motion. No one has time for that — so we make a computer do it.

I'm going to assume you get the basic idea of a numerical calculation. However, if you want a very basic intro — here's a video for you.

OK, back to our elastic collision. The key to doing this numerically (breaking into short time intervals) is to calculate the force on each object. If we know the force, we can update the momentum and then update the position. But how do you get the force from a collision?

The answer is to use a spring. Imagine that we have two balls that collide. We can approximate the force during the collision using a spring. This spring will push the two balls apart with a magnitude that is proportional to the distance that the balls overlap.

Press enter or click to view image in full size



Here the vector $\vec{r}$ is the displacement from ball A to ball B and R is the radius of the two balls. So, if the magnitude of $\vec{r}$ is less than $2R$ there will be a collision. The value k is a fake spring constant that describes the “stiffness” of the ball interactions. Oh, that $\hat{r}$ is just a unit vector to make the force a vector. The equation above is for the force on ball A — the force on ball B is just the negative of that force. One trick is to check and see if the balls overlap. If they don't overlap, there's no force.

Great. Let's do it. I'm not going to go over ALL the code, [but it's here if you want to play with it](#). I'm using [Web VPython](#), so that includes

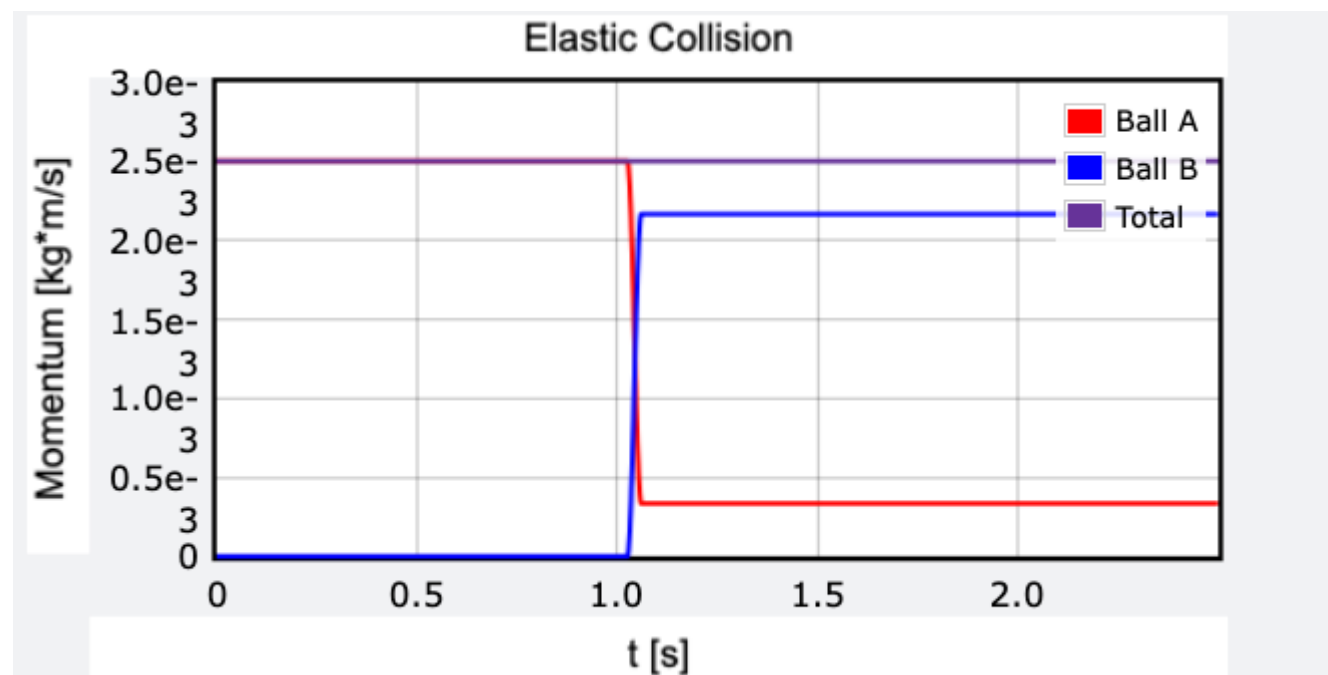
functions to create 3D object — in this case 3D balls. Here's my elastic collision for two equal mass balls with one initially stationary.



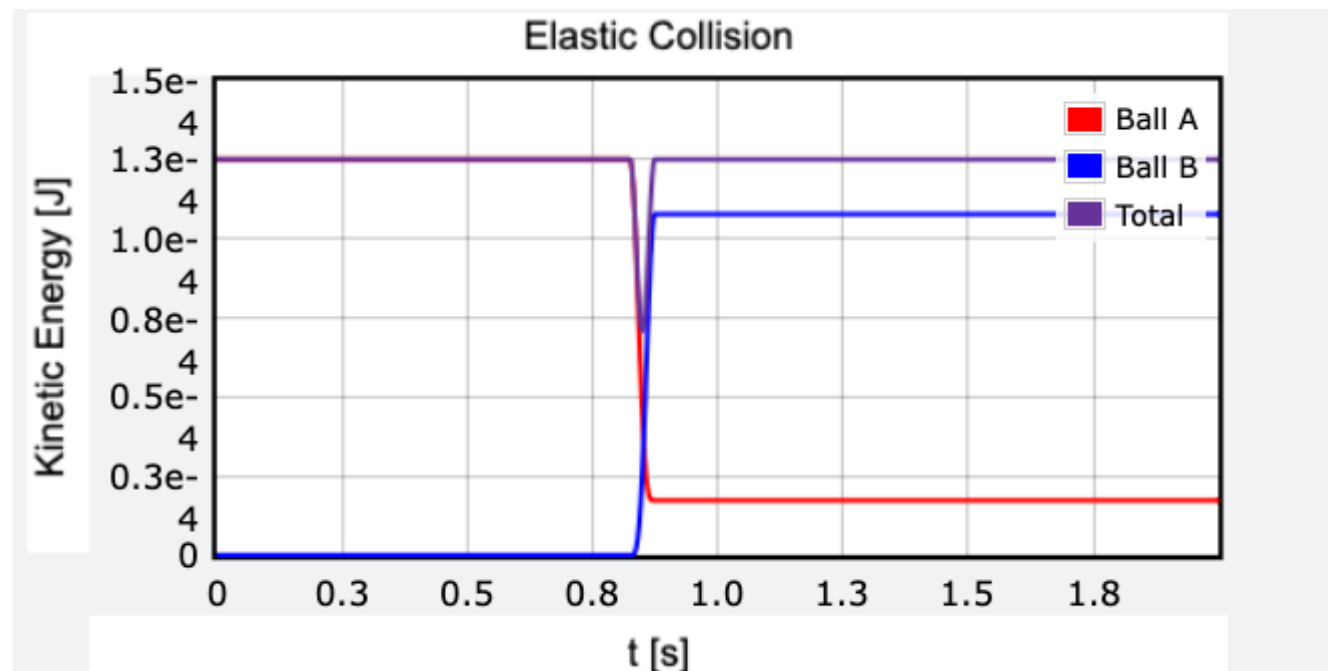
Check that out. I set up the balls so they it was a “glancing” collision and it looks like it works just fine. This is because the “r-vector” is from center of ball A to the center of ball B. This means the impact force has a vertical component.

OK, it looks cool — but is it correct? If it's a realistic elastic collision, we need to conserve both momentum and kinetic energy. Here is a plot of the momentum in the x-direction for this motion.

Press enter or click to view image in full size



That looks good. What about the kinetic energy?
Press enter or click to view image in full size



It might look like the kinetic energy is conserved, but it's actually not. Look at the purple line — it drops down a little during the collision and then pops back up. This is because there is a spring. During the collision this fake spring gets compressed and stores some elastic potential energy. So, energy is still conserved — just not kinetic energy. But the total KE before and after the collision is still constant — so, it works.

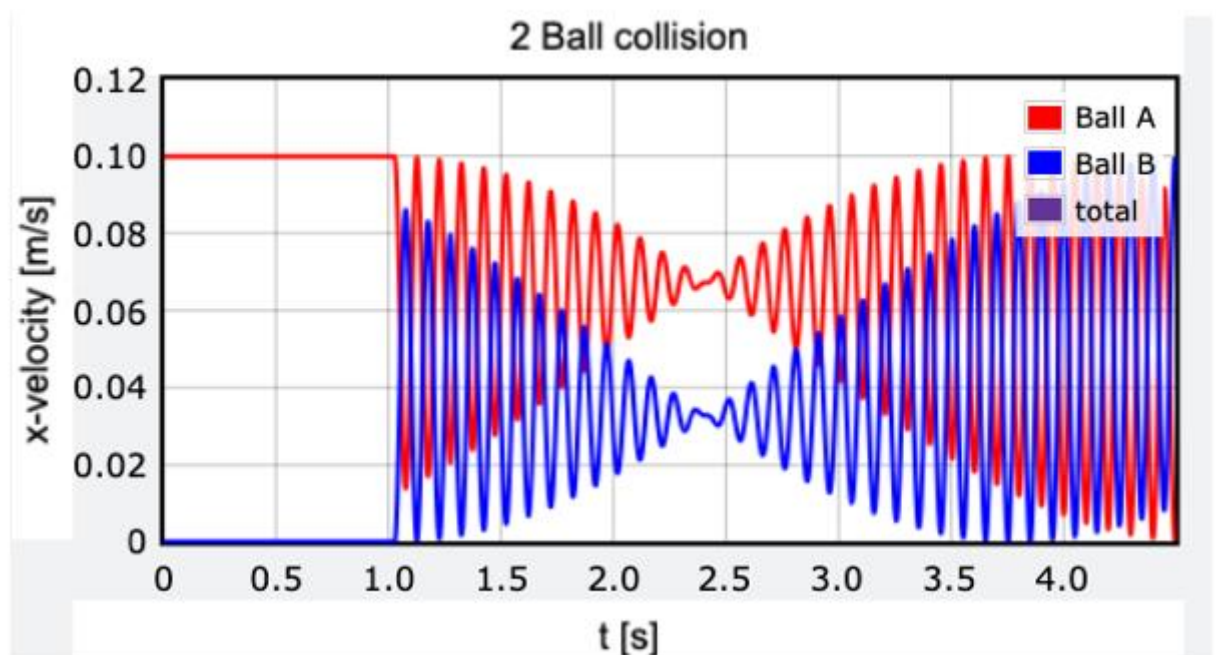
Perfectly Inelastic Collision

Let's move to the next collision. We can still use a fake spring to model the impact force, but how do we make the two balls stick together? Here's another collision trick. I'm going to again use a spring force to push the balls apart when they overlap. However, in this case the spring won't stop acting on the balls after the collide. When the balls are no longer overlapping, the fake spring will pull them back together. If they are stuck together then they will have to have the same speed.

[Here's the code for the following animation.](#)



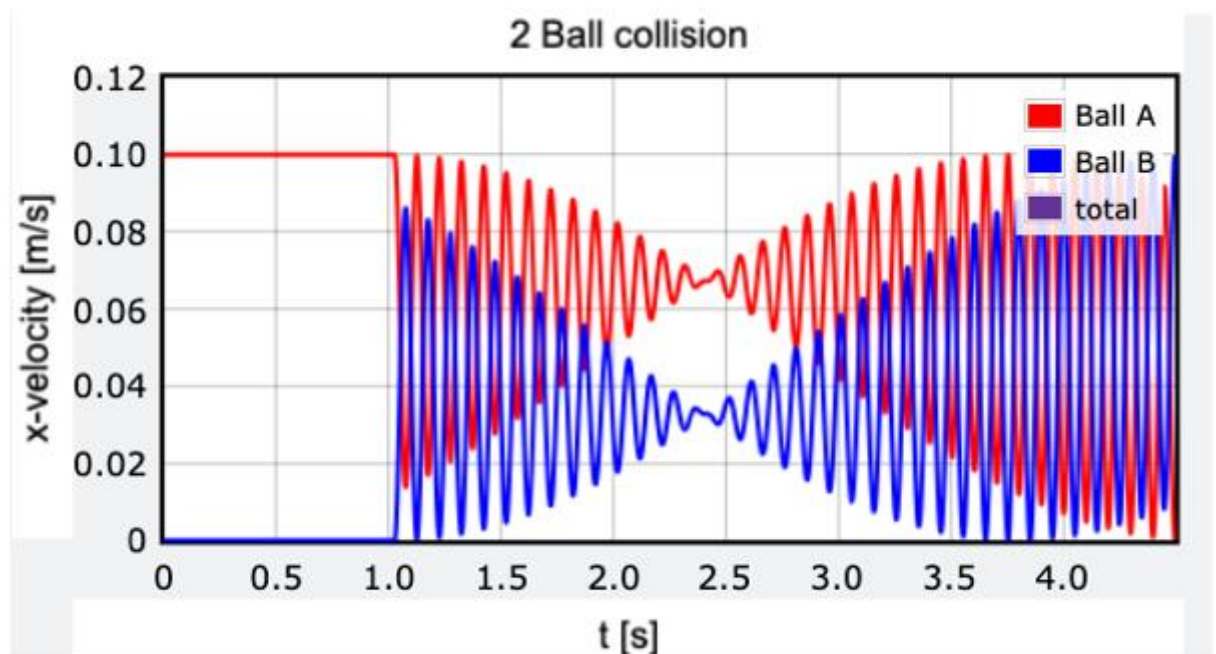
I like it. I mean, you can see that they don't perfectly "stick together" but it basically does the trick. We could check that momentum is conserved — but we really want to see the velocity. Here's a plot of the velocity in the x-direction for both balls.



They don't have the same velocity so it's not actually an inelastic collision. Can we fix this? Yes we can. I want to make this oscillation die down over time. I could add a velocity dependent drag force to the balls, but then the two balls would have the same velocity as they stop. I don't want them to stop all together. The trick is to add a force that

only depends on the relative velocity of the two balls. That means that if they are moving with the same velocity there won't be a drag force.

Check it out.



Nice. Oh, momentum IS conserved as long as the drag force on one ball is equal and opposite to the force on the other ball. This will happen because we are using the relative velocity.

Plain Collisions

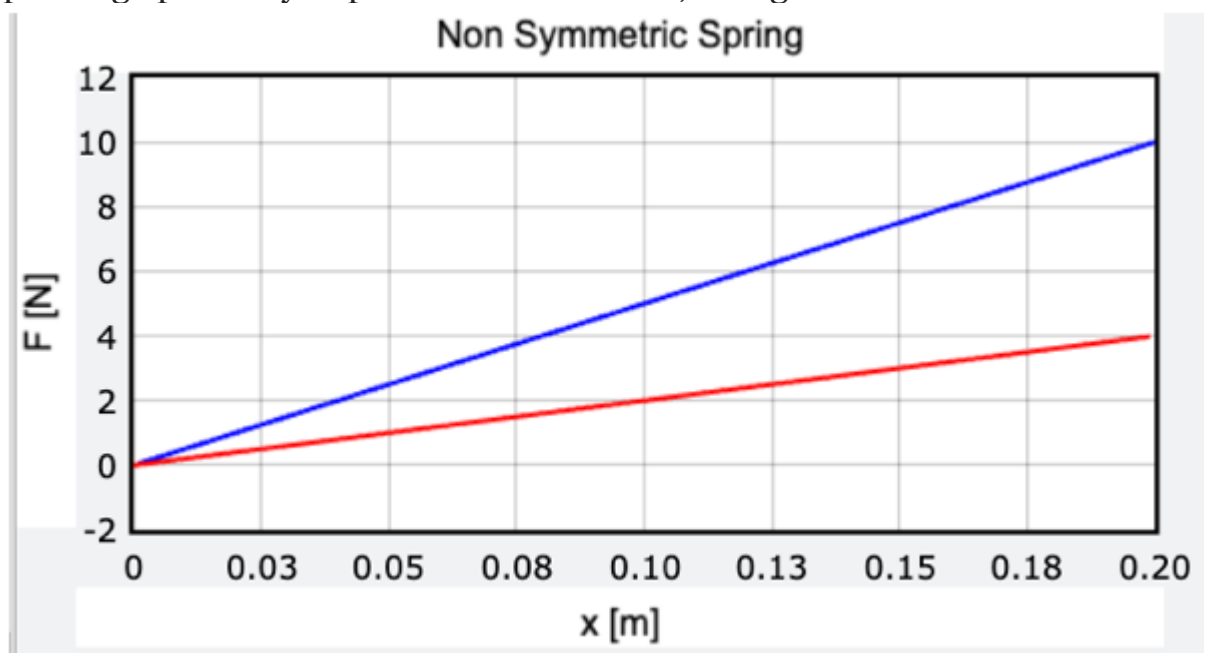
Are we going to need another trick here? Yup. Remember, plain collisions need to conserve momentum but NOT conserve kinetic energy. Yes, the inelastic collisions did this but the ball stuck together. I want them to hit and bounce off without conserving KE.

Let's think about springs. Of course we are going to use a fake spring again. Imagine you have a ball colliding with a spring. The more the spring is compressed, the greater the spring force (that's Hooke's Law). We could use this increasing spring force to find the work done by the spring (which would be the negative of the energy stored in the spring). We get the following expression.

$$\Delta U_s = \frac{1}{2} k x^2$$

Here k is the spring constant (again) and x is the distance the spring is compressed. Once the spring is compressed and the ball stops, this spring potential energy is converted back to kinetic energy and the ball will have the same final speed.

But what if we have a non-symmetrical spring? One where the spring constant for compression is greater than the spring constant for pushing apart? If you plot force vs distance, it might look like this.



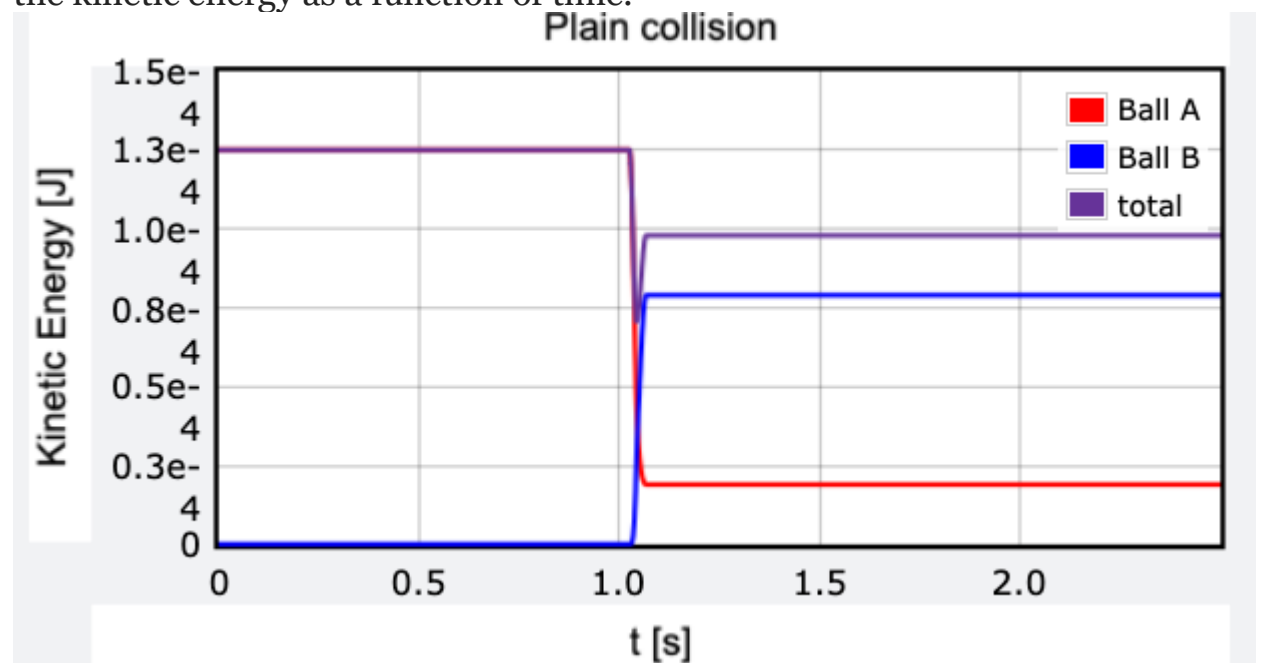
When a ball pushes against the spring, it would be the work done by the blue line. For the ball speeding up, it would be the red curve. With a weaker repelling spring, the ball would bounce back with a lower velocity.

But what about momentum? As long as the force on the two balls is equal and opposite, momentum will be conserved. OK, let's try

it. [Here's the code.](#) The one python trick is to figure out if the two balls are moving TOWARDS or AWAY from each other — but it's not that tough.



This might look like the elastic collision, but it's not. Here's a plot of the kinetic energy as a function of time.



You can change the elasticity (I made up that word) of the collision by changing the relative values of the two fake spring constants (for compression vs. repelling).

There you have it — python models for the three types of collisions.

Physics